

<



Portfolio Phase 3

Internationale Hochschule Duales Studium

Studiengang: Informatik

RideAndPark - Finalisierungsphase

Lukas Penner

Matrikelnummer: 4234713

Elsternhorst 13, 35415 Pohlheim

Betreuer/in: Armands Strazds

Abgabe: 28.05.2026

Inhaltsverzeichnis

1.	Zielsetzung, Idee und Konzept.....	1
2.	Quellen, Ressourcen und Software	1
2.1.	Quellen und Ressourcen.....	1
2.2.	Verwendete Software und Technologien	1
2.3.	Zentrale Backend-Dateien	2
3.	Breakdown der Umsetzung.....	2
3.1.	Architektur.....	2
3.2.	API-Endpunkte.....	3
3.3.	Datenverarbeitung und internes Datenmodell	5
3.4.	Cache-Strategie und Performance	6
3.5.	Fehlerbehandlung und Fallback-Mechanismen.....	7
3.6.	Versionsverwaltung und Entwicklungsworkflow.....	9
4.	Ergebnis von der Konzeptidee bis zur Umsetzung.....	9
5.	Zielerreichung.....	10
6.	Reflexion der Leistung	11
7.	Fazit	12

1. Zielsetzung, Idee und Konzept

Ziel des Projekts RideAndPark ist die Entwicklung einer Webanwendung, die freie Parkplätze in der Nähe eines Zielortes sichtbar macht. Nutzer sollen aktuelle Parkplatzinformationen über eine zentrale Anwendung abrufen können, ohne direkt mit unterschiedlichen externen Datenquellen arbeiten zu müssen.

Der Schwerpunkt meiner Arbeit liegt auf dem Backend. Es übernimmt die Aufgabe, externe Parkplatzdaten abzurufen, zu vereinheitlichen, zu filtern und dem Frontend über eine eigene REST-API bereitzustellen. Dadurch bleibt das Frontend unabhängig von der Struktur der externen Parkplatz-API und kann mit einem stabilen internen Datenmodell arbeiten.

Das Konzept aus Phase 1 sah eine Client-Server-Architektur vor, bei der das Backend als zentrale Verarbeitungsschicht zwischen Frontend und externer Datenquelle steht. In Phase 2 wurde diese Architektur praktisch umgesetzt. In Phase 3 wurde das Backend finalisiert, stabilisiert und anhand der ursprünglichen Ziele bewertet.

Methodisch wurde ein API-first-Ansatz verfolgt. Zuerst wurden die benötigten Schnittstellen und Datenstrukturen definiert, danach wurden die Verarbeitungsschichten im Backend aufgebaut. Die Umsetzung erfolgte iterativ und wurde über ein Kanban-Board organisiert.

2. Quellen, Ressourcen und Software

2.1. Quellen und Ressourcen

- MobiData BW ParkAPI als externe Quelle für Parkplatzdaten
- OpenStreetMap / Nominatim für Geocoding von Zielorten
- Eigene Architekturdiagramme aus Phase 1 und Phase 2
- Eigene Projektplanung und Aufgabenverwaltung über ein Kanban-Board
- Projektdokumentation aus Phase 1 und Phase 2
- README- und MVP-Dokumentation des Projekts
- GitHub-Repository zur Versionsverwaltung und Teamkoordination

2.2. Verwendete Software und Technologien

- Node.js als Laufzeitumgebung
- Express.js für die REST-API
- JavaScript im CommonJS-Modulformat

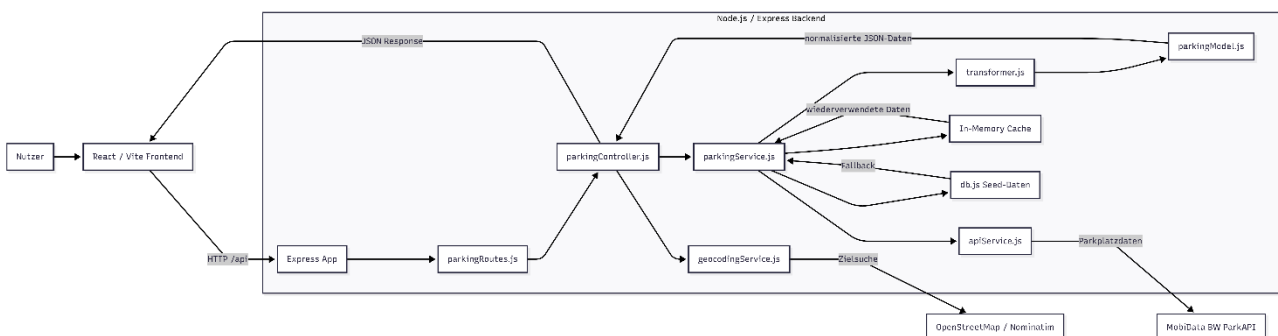
- dotenv für Umgebungsvariablen
- cors für die Konfiguration erlaubter Frontend-Zugriffe
- Node Test Runner für automatisierte Backend-Tests
- Docker und Docker Compose für containerisierte Ausführung
- Git und GitHub für Versionsverwaltung
- React/Vite im Frontend als API-Konsument

2.3. Zentrale Backend-Dateien

- src/app.js – Express-App, Middleware, API-Mounting und Fehlerbehandlung
- src/server.js – Start des Backend-Servers
- src/routes/parkingRoutes.js – Definition der API-Endpunkte
- src/controllers/parkingController.js – Validierung von Requests und HTTP-Verarbeitung
- src/services/parkingService.js – Fachlogik, Caching, Filterung und Fallback
- src/services/apiService.js – Abruf externer Parkplatzdaten
- src/services/geocodingService.js – Geocoding über Nominatim
- src/utlis/transformer.js – Vereinheitlichung externer Rohdaten
- src/models/parkingModel.js – Internes Parkplatzmodell
- src/config/db.js – Lokale Seed-Daten für den Fallback

3. Breakdown der Umsetzung

3.1. Architektur



Das Backend wurde in mehrere Schichten aufgeteilt:

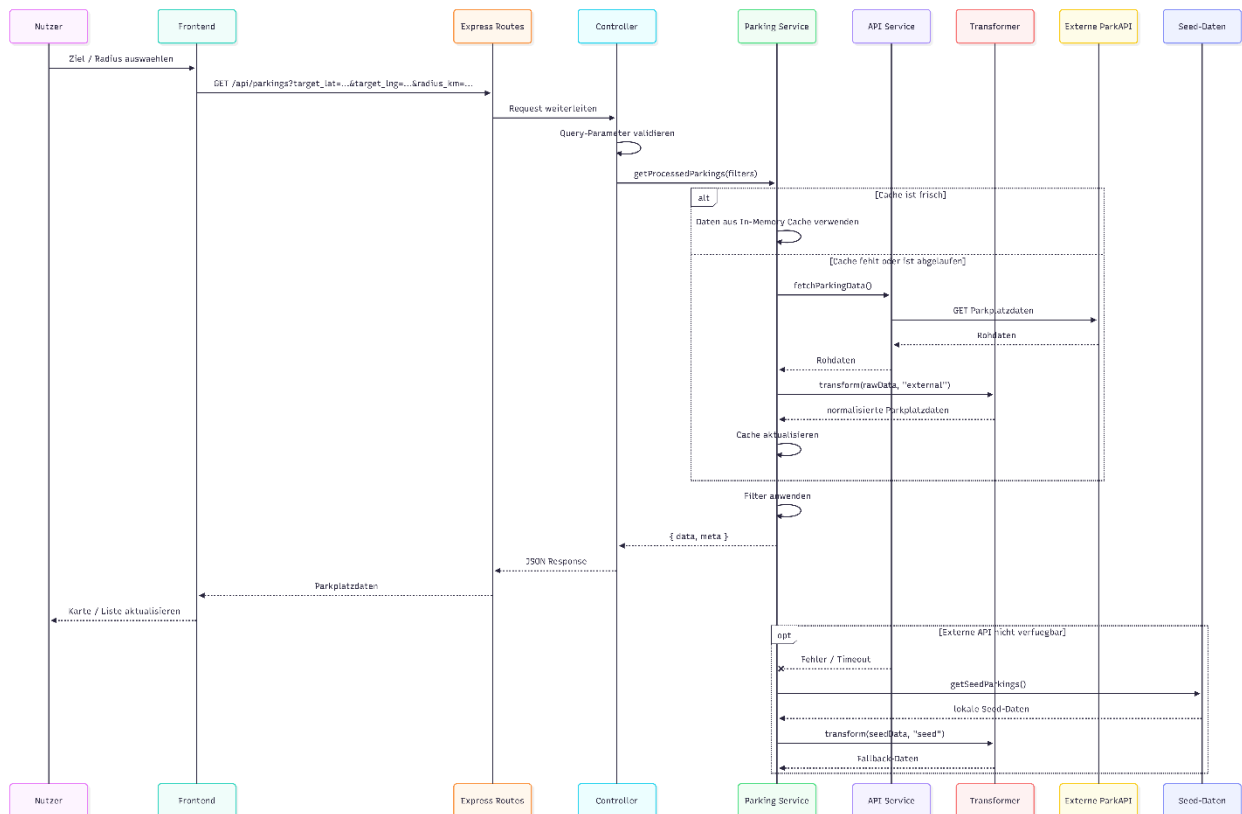
- Routes definieren die verfügbaren Endpunkte

- Controller prüfen und validieren eingehende Parameter
- Services enthalten die fachliche Logik
- Der Transformer normalisiert externe Daten
- Das Modell beschreibt die interne Parkplatzstruktur

Diese Schichtenstruktur erhöht die Wartbarkeit, weil Verantwortlichkeiten klar getrennt sind. Änderungen an der externen API müssen dadurch hauptsächlich im API-Service oder Transformer behandelt werden und wirken sich nicht direkt auf das Frontend aus.

Das Frontend kommuniziert ausschließlich mit der eigenen REST-API. Externe Dienste werden nur vom Backend angesprochen. Dadurch können Validierung, Fehlerbehandlung, Caching und Datenverarbeitung zentral umgesetzt werden.

3.2. API-Endpunkte



Final bereitgestellt wurden folgende Backend-Endpunkte:

Methode	Endpoint	Zweck
GET	/api/health	Healthcheck des Backends
GET	/api/parkings	Abruf und Filterung von Parkplätzen

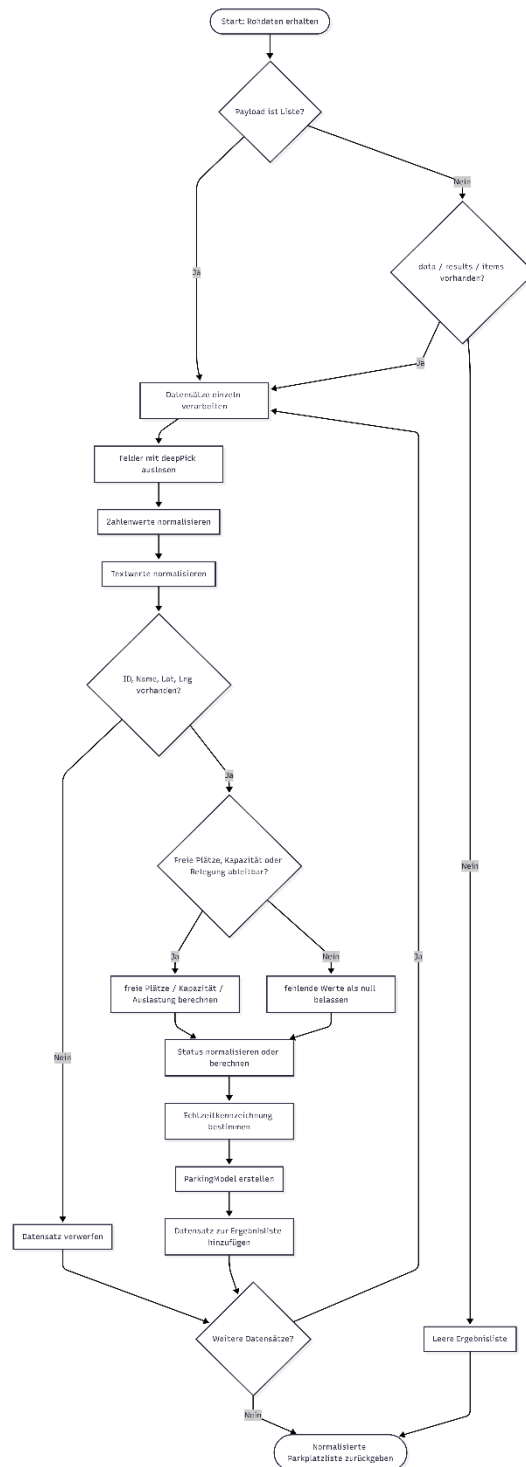
Methode	Endpunkt	Zweck
GET	/api/parkings/:id	Abruf eines einzelnen Parkplatzes
POST	/api/parkings/refresh	Manuelle Aktualisierung der Parkplatzdaten
GET	/api/geocode	Umwandlung eines Zielortes in Koordinaten
GET	/api/statistics	Rückgabe aggregierter Parkplatzstatistiken

Der wichtigste Endpunkt ist /api/parkings, da er die Grundlage für die Karten- und Listenansicht im Frontend bildet. Unterstützt werden unter anderem:

- Radiusfilter,
- Zielkoordinaten,
- Name,
- Quelle,
- Echtzeitstatus,
- sowie offene Parkplätze.

Die serverseitige Filterung reduziert die Datenmenge, die an das Frontend übertragen werden muss.

3.3. Datenverarbeitung und internes Datenmodell



Externe Parkplatzdaten werden nicht unverändert an das Frontend weitergegeben. Stattdessen übernimmt der Transformer die Vereinheitlichung in ein internes Datenmodell.

Das Modell enthält unter anderem:

- Parkplatz-ID
- Name

- Koordinaten
- freie Stellplätze
- Gesamtkapazität
- Auslastung
- Status
- Quelle
- Echtzeitkennzeichnung
- Aktualisierungszeitpunkt
- Öffnungszeiten

Unterschiedliche Feldnamen externer APIs werden dadurch normalisiert. Das Frontend arbeitet dadurch unabhängig von der jeweiligen Datenquelle.

Die Statuswerte werden auf ein einheitliches Schema reduziert:

- open
- limited
- full
- unknown

Falls die externe API keinen eindeutigen Status liefert, berechnet das Backend den Status ersatzweise aus freien Stellplätzen, Kapazität oder Auslastung.

Unvollständige Datensätze ohne Koordinaten, Namen oder IDs werden verworfen, damit nur valide Daten an das Frontend weitergegeben werden.

3.4. Cache-Strategie und Performance

Zur Reduzierung wiederholter externer API-Aufrufe wurde ein In-Memory-Cache implementiert.

Die Cache-Dauer ist über `PARKING_CACHE_TTL_MS` konfigurierbar. Standardmäßig werden Daten für zehn Minuten wiederverwendet.

Dadurch muss die externe API nicht bei jeder Nutzeranfrage erneut kontaktiert werden.

Zusätzlich verhindert eine `pendingLoad`-Logik parallele Mehrfachanfragen: Wenn mehrere Requests gleichzeitig eingehen, während bereits ein externer Abruf läuft, wird derselbe Ladevorgang wiederverwendet.

Besonders hilfreich war diese Optimierung bei mehreren parallelen Frontend-Anfragen. Ohne Caching und pendingLoad entstanden wiederholte externe API-Aufrufe mit unnötiger zusätzlicher Verarbeitung.

Die Filterung erfolgt ausschließlich serverseitig auf den bereits transformierten Daten. Dadurch enthält das Frontend weniger eigene Logik und kann die Ergebnisse direkt darstellen.

3.5. Fehlerbehandlung und Fallback-Mechanismen

Die größte Herausforderung bestand in der Abhängigkeit von externen Echtzeitdaten. Antwortzeiten der externen API waren teilweise instabil und einzelne Datensätze enthielten unvollständige Informationen.

Externe API-Aufrufe wurden deshalb über try-catch-Blöcke abgesichert.

```
// Beispiel: API-Aufruf mit Timeout
async function fetchExternalData(url) {
  const controller = new AbortController();

  const timeout = setTimeout(() => {
    controller.abort();
  }, 5000);

  try {
    const response = await fetch(url, {
      signal: controller.signal
    });

    if (!response.ok) {
      throw new Error(`Externe API antwortete mit Status ${response.status}`);
    }

    const data = await response.json();
    return data;
  } catch (error) {
    throw error;
  } finally {
    clearTimeout(timeout);
  }
}
```

Der try-Block enthält den eigentlichen API-Aufruf. Falls die externe API nicht erreichbar ist, zu lange braucht oder einen fehlerhaften HTTP-Status zurückgibt, wird ein Fehler ausgelöst. Im finally-Block wird der Timeout immer aufgeräumt, egal ob der Aufruf erfolgreich war oder fehlgeschlagen ist.

```
// Beispiel: Retry-Logik mit Fallback auf lokale Seed-Daten
async function getParkingData() {
  try {
    return await fetchExternalData("https://example.com/api/parking");
  } catch (firstError) {
    // Fallback-Logik hier
  }
}
```

```

    console.warn("Erster API-Aufruf fehlgeschlagen, Retry wird
versucht.");

    try {
        return await fetchExternalData("https://example.com/api/parking");
    } catch (secondError) {
        console.error("Retry fehlgeschlagen, Seed-Daten werden verwendet.");

        return seedParkingData;
    }
}
}

```

Hier wird zuerst versucht, Daten von der externen API zu laden. Wenn der erste Versuch fehlschlägt, wird ein zweiter Versuch durchgeführt. Erst wenn auch dieser Retry fehlschlägt, greift das Backend auf lokale Seed-Daten zurück. Dadurch bleibt die REST-API weiterhin erreichbar, auch wenn die externe Datenquelle temporär ausfällt.

```

// Beispiel: Route mit try-catch und Weitergabe an zentrale
Fehlerbehandlung
app.get("/api/parking", async (req, res, next) => {
    try {
        const data = await getParkingData();

        res.json({
            success: true,
            data
        });
    } catch (error) {
        next(error);
    }
});

```

In der Route wird der Datenabruf ebenfalls durch try-catch abgesichert. Falls ein unerwarteter Fehler entsteht, wird er mit next(error) an die zentrale Express-Fehlerbehandlung weitergegeben.

```

// Beispiel: Prüfung ungültiger Query-Parameter
app.get("/api/stations", async (req, res, next) => {
    try {
        const limit = Number(req.query.limit);

        if (req.query.limit && Number.isNaN(limit)) {
            return res.status(400).json({
                success: false,
                error: "Ungültiger Query-Parameter: limit muss eine Zahl sein."
            });
        }

        const active = req.query.active;

        if (active && active !== "true" && active !== "false") {
            return res.status(400).json({
                success: false,

```

```

        error: "Ungültiger Query-Parameter: active muss true oder false
sein."
    });
}

res.json({
    success: true,
    data: []
});
} catch (error) {
    next(error);
}
});

```

Dieses Snippet zeigt, wie ungültige Query-Parameter direkt mit HTTP-Statuscode 400 Bad Request beantwortet werden. Dadurch werden fehlerhafte Zahlenwerte oder ungültige Boolean-Werte früh abgefangen.

3.6. Versionsverwaltung und Entwicklungsworkflow

Für die Versionsverwaltung wurde GitHub mit einer einfachen Branching-Strategie verwendet.

Der main-Branch enthält ausschließlich stabile Entwicklungsstände. Neue Funktionen und größere Änderungen wurden zunächst im develop-Branch integriert und getestet.

Zusätzlich wurden separate Branches für einzelne Verantwortungsbereiche verwendet:

- frontend für die React-Oberfläche
- backend für die API- und Service-Entwicklung
- test für automatisierte Tests und experimentelle Änderungen

Dadurch konnten Backend-, Frontend- und Testentwicklungen voneinander getrennt umgesetzt werden, ohne stabile Zwischenstände im Hauptbranch zu beeinflussen.

Änderungen wurden nach erfolgreicher Integration kontrolliert in den develop-Branch und anschließend in den main-Branch übernommen.

Das Projekt-Repository ist unter folgendem Link verfügbar:

<https://github.com/RideAndPark/RideAndPark>

4. Ergebnis von der Konzeptidee bis zur Umsetzung

Die ursprüngliche Konzeptidee aus Phase 1 war ein Backend, das externe Parkplatzdaten integriert, verarbeitet und über eine REST-API für das Frontend bereitstellt. Dieses Ziel wurde erreicht.

Das finale Backend stellt eine funktionierende REST-API für Parkplatzdaten bereit und unterstützt zusätzlich Geocoding-, Statistik- und Healthcheck-Endpunkte. Externe Daten werden serverseitig

validiert, transformiert und gefiltert. Ergänzend wurden Caching-Mechanismen, Fallback-Daten sowie automatisierte Tests integriert, um die Stabilität des Systems zu erhöhen.

Die in Phase 2 beschriebenen Schwerpunkte wurden weitgehend umgesetzt:

- Schichtenarchitektur,
- API-Anbindung,
- Transformationslogik,
- Fehlerbehandlung,
- Caching,
- sowie zentrale Datenverarbeitung.

Das Ergebnis entspricht damit dem Ziel der Arbeit.

Nicht vollständig umgesetzt wurden:

- persistente Datenbankanbindung,
- Redis-basierter verteilter Cache,
- umfangreiches Monitoring,
- produktionsreife Skalierungsmechanismen.

Diese Punkte wurden bewusst nicht Bestandteil des MVPs.

Statt einer Datenbank wurde für die aktuelle Projektstufe ein In-Memory-Cache mit lokalen Fallback-Daten verwendet. Für die Anforderungen des MVPs reicht diese Lösung aus. Für einen produktiven Mehrinstanzbetrieb wäre eine persistente Infrastruktur notwendig.

5. Zielerreichung

Das Backend erfüllt die zentralen Anforderungen des Projekts. Parkplatzdaten können über eine eigene REST-API abgerufen und serverseitig normalisiert werden. Zusätzlich unterstützt das System Radiusfilter, Zielkoordinaten sowie die Verarbeitung von Echtzeitdaten. Die Anwendung ist sowohl lokal als auch containerisiert über Docker ausführbar.

Die wichtigste Zielabweichung betrifft die ursprünglich angedachte Datenbank.

In der finalen Umsetzung wird keine persistente Datenbank verwendet. Der Grund dafür ist die Fokussierung auf ein MVP. Für die benötigte Live-Abfrage und kurzfristige Zwischenspeicherung reicht der In-Memory-Cache aus.

Eine Datenbank wäre insbesondere relevant für:

- historische Analysen,
- langfristige Datenspeicherung,
- sowie produktive Skalierung.

Auch die Performanceanalyse wurde nur teilweise umgesetzt. Es existieren bereits:

- Caching,
- Retry-Mechanismen,
- reduzierte externe API-Aufrufe,
- sowie grundlegende Optimierungen.

Ein systematisches Monitoring mit Laufzeitmessungen wurde jedoch noch nicht integriert.

6. Reflexion der Leistung

Der Bearbeitungsprozess zeigt, dass die Trennung zwischen Frontend, Backend und externer Datenquelle sinnvoll ist. Besonders bei heterogenen Echtzeitdaten ermöglicht eine zentrale Verarbeitungsschicht kontrollierte Validierung, Fehlerbehandlung und Datenvereinheitlichung.

Eine wichtige Erkenntnis war, dass externe APIs nicht nur technisch angebunden werden müssen.

Probleme entstanden insbesondere durch uneinheitliche Feldnamen, fehlende Statusinformationen, instabile Antwortzeiten sowie unterschiedliche Datenstrukturen der externen APIs. Dadurch musste die Transformationslogik deutlich flexibler aufgebaut werden als ursprünglich geplant.

Die größte technische Herausforderung bestand in der Verarbeitung heterogener API-Antworten. Einzelne Statuswerte mussten teilweise aus vorhandenen Kapazitätsdaten berechnet werden, weil die externe API keine konsistenten Statusinformationen bereitstellte.

Besonders hilfreich war die Kombination aus:

- In-Memory-Caching,
- Retry-Mechanismen,
- Timeouts,
- und lokalen Fallback-Daten.

Dadurch konnten unnötige externe Requests reduziert und temporäre API-Ausfälle abgefangen werden.

Rückblickend wäre eine frühere systematische Performanceanalyse sinnvoll gewesen, um Engpässe präziser identifizieren zu können.

Positiv ist, dass das Backend modular aufgebaut wurde. Neue Datenquellen, zusätzliche Filter oder eine spätere Datenbankanbindung können ergänzt werden, ohne die gesamte Architektur umzubauen.

Die automatisierten Tests erleichtern zukünftige Erweiterungen und erhöhen die Nachvollziehbarkeit zentraler Backend-Funktionen.

7. Fazit

Die Finalisierungsphase zeigt, dass aus der ursprünglichen Konzeptidee ein funktionsfähiges Backend für RideAndPark entstanden ist.

Die Anwendung verfügt über:

- eine eigene REST-API,
- eine klare Schichtenarchitektur,
- externe Datenanbindung,
- Transformationslogik,
- Geocoding,
- Caching,
- Fehlerbehandlung,
- Fallback-Mechanismen,
- sowie automatisierte Tests.

Das Backend erreicht damit die Zielsetzung der Arbeit für den definierten MVP-Umfang.

Es stellt dem Frontend eine stabile Grundlage bereit, um Parkplätze in der Nähe eines Zielortes anzuzeigen und nach relevanten Kriterien zu filtern.

Für eine Weiterentwicklung wären insbesondere folgende Punkte sinnvoll:

- persistente Speicherung in einer Datenbank
- Redis oder vergleichbarer verteilter Cache
- systematisches Monitoring und Laufzeitmessungen
- erweiterte Tests mit realen API-Payloads
- produktionsnahe Deployment- und Sicherheitskonfiguration

Insgesamt konnte das Backend von der Konzeption über die Erarbeitung bis zur Finalisierung erfolgreich umgesetzt werden. Die ursprüngliche Zielsetzung wurde erreicht, während bewusst ausgelassene Produktivfunktionen mögliche nächste Entwicklungsschritte darstellen.